

```

•   $route () {
•     this.selectMovie()
•   }
• },
• methods: {
•   selectMovie () {
•     this.selectedMovie = movies[this.$route.params.id]
•   }
• }
• }

```

Have a look at this snippet here.

```

• data () {
•   return {
•     selectedMovie: movies[this.$route.params.id]
•   }
• }

```

We've used the `$route.params.id` to refer to a dynamic element from our Global Movies Store. This was done by setting `selectedMovie` property within the component to dynamically read the movies data.

```

• watch: {
•   $route () {
•     this.selectMovie()
•   }
• },
• methods: {
•   selectMovie () {
•     this.selectedMovie = movies[this.$route.params.id]
•   }
• }

```

We now set up a `watch` property to observe if there are any changes in the router. Upon detecting one, we'll call the `selectMovie` method to pull the updated data and render the same. This will handle the component updates when a user surfs through different movies.

Our routes should now work properly. Click on each movie in the footer to see if the title in the `Movie` component changes to pull the name of the movie that has been selected.